

Docker Volume Persistence Practical Documentation

This document provides a detailed, step-by-step guide for the Docker Volume Persistence Practical, explaining each command and its purpose. The content is based on the provided practical notes, with minor corrections for accuracy as highlighted in the original text.

Docker Installation and Setup

Step 1: Switch to Root User

```
sudo su
```

Explanation: This command switches the current user to the root user. `sudo` allows a permitted user to execute a command as the superuser or another user, and `su` (substitute user) without any arguments defaults to the root user. This is often necessary for system-level installations and configurations.

Step 2: Update Packages

```
yum update -y
```

Explanation: This command updates all installed packages on the system using `yum`, a package manager for RPM-based Linux distributions (like CentOS, RHEL, Fedora). The `-y` flag automatically answers yes to any prompts, allowing for non-interactive updates.

Step 3: Install Docker

```
yum install docker -y
```

Explanation: This command installs the Docker engine package using `yum`. The `-y` flag ensures that the installation proceeds without requiring manual confirmation.

Step 4: Verify Docker Installation

```
docker --version
```

Explanation: This command displays the installed Docker version, confirming that Docker has been successfully installed on the system.

Expected Output:

```
Docker version xx.xx.x
```

Step 5: Check Docker Binary

```
which docker
```

Explanation: This command shows the full path of the Docker executable. It helps confirm that the `docker` command is recognized and where its binary is located.

Expected Output:

```
/usr/bin/docker
```

Step 6: Check Docker Service

```
systemctl status docker
```

Explanation: This command checks the current status of the Docker daemon service. It shows whether the service is active, inactive, enabled, or disabled, along with recent logs.

Step 7: Start Docker

```
systemctl start docker
```

Explanation: This command starts the Docker daemon service. The Docker daemon must be running for any Docker commands to function.

Step 8: Enable Docker at Boot

```
systemctl enable docker
```

Explanation: This command configures the Docker service to start automatically every time the system boots up. This ensures that Docker is always available after a system restart.

Step 9: Verify Docker Service

```
systemctl status docker
```

Explanation: After starting and enabling, this command re-checks the Docker service status to confirm it is `active (running)` and enabled for future boots.

Expected Output:

```
Active: active (running)
```

Docker Image Management

Step 10: Download Docker Images

```
docker pull nginx:latest
docker pull ubuntu:latest
```

Explanation: These commands download the `nginx` and `ubuntu` Docker images from Docker Hub. `nginx:latest` pulls the latest version of the Nginx web server image, and `ubuntu:latest` pulls the latest Ubuntu base image. These images will be used as base images for custom containers.

Step 11: Verify Images

```
docker images
```

Explanation: This command lists all Docker images that are locally stored on the system. It confirms that the `nginx` and `ubuntu` images have been successfully downloaded.

Example Output:

REPOSITORY	TAG	IMAGE ID
nginx	latest	<image_id>
ubuntu	latest	<image_id>

Project Setup and Dockerfile Creation

Step 12: Create Project Directory

```
mkdir restaurant-project
cd restaurant-project
```

Explanation: These commands create a new directory named `restaurant-project` and then change the current working directory into it. This directory will house all the project files, including the Dockerfile and web content.

Step 13: Project Structure

```
restaurant-project/  
├── Dockerfile  
├── index.html  
└── style.css
```

Explanation: This section illustrates the expected file structure within the `restaurant-project` directory. It indicates that there will be a `Dockerfile`, an `index.html` file, and a `style.css` file.

Step 14: Create Dockerfile

```
vi Dockerfile
```

Explanation: This command opens the `Dockerfile` in the `vi` text editor. The `Dockerfile` contains instructions for Docker to build a custom image. Note the capital 'D' in `Dockerfile` as filenames are case-sensitive in Linux.

Content to Paste:

```
FROM nginx:latest

WORKDIR /usr/share/nginx/html

COPY index.html .
COPY style.css .

VOLUME ["/restaurant-data"]

EXPOSE 80

CMD ["nginx","-g","daemon off;"]
```

Explanation of Dockerfile commands:

- `FROM nginx:latest` : Specifies `nginx:latest` as the base image for this Docker image.
- `WORKDIR /usr/share/nginx/html` : Sets the working directory inside the container to `/usr/share/nginx/html` . Subsequent commands will be executed relative to this directory.
- `COPY index.html .` : Copies the `index.html` file from the host (current directory) into the current working directory of the container.
- `COPY style.css .` : Copies the `style.css` file from the host into the current working directory of the container.
- `VOLUME [ "/restaurant-data" ]` : Creates a Docker volume named `restaurant-data` . This volume will be used to persist data generated by the container, even if the container is removed.
- `EXPOSE 80` : Informs Docker that the container listens on network port 80 at runtime. This is a documentation step and doesn't actually publish the port.
- `CMD [ "nginx", "-g", "daemon off;" ]` : Specifies the command to run when the container starts. Here, it starts the Nginx web server in the foreground.

Save:

```
:wq
```

Step 15: Create index.html

```
vi index.html
```

Explanation: This command opens the `index.html` file in the `vi` text editor. This file will contain the HTML content for the web page served by the Nginx container.

Example Content:

```
<!DOCTYPE html>
<html>
<head>
<title>Restaurant</title>
<link rel="stylesheet" href="style.css">
</head>

<body>

<h1>Welcome to My Restaurant</h1>

<p>Docker Volume Practical</p>

</body>
</html>
```

Save:

```
:wq
```

Step 16: Create style.css

```
vi style.css
```

Explanation: This command opens the `style.css` file in the `vi` text editor. This file will contain the CSS rules to style the `index.html` page.

Example Content:

```
body{  
  
background:#f5f5f5;  
  
text-align:center;  
  
font-family:Arial;  
  
}  
  
h1{  
  
color:red;  
  
}
```

Save:

```
:wq
```

Building and Running the Docker Container

Step 17: Build Docker Image

```
docker build -t restaurant:v1 .
```

Explanation: This command builds a Docker image named `restaurant` with the tag `v1` using the `Dockerfile` in the current directory (`.`). This process executes the instructions in the Dockerfile to create a new image.

Step 18: Verify Image

```
docker images
```

Explanation: This command lists all local Docker images, confirming that the `restaurant:v1` image has been successfully built and is available.

Expected Output:

REPOSITORY	TAG	IMAGE ID
restaurant	v1	<image_id>

Step 19: Create Docker Volume

```
docker volume create restaurant-data
```

Explanation: This command explicitly creates a Docker volume named `restaurant-data`. This volume will be used to store persistent data that can be shared between containers or persist even after containers are removed.

Step 20: List Volumes

```
docker volume ls
```

Explanation: This command lists all Docker volumes on the system, verifying that `restaurant-data` has been created.

Step 21: Inspect Volume

```
docker volume inspect restaurant-data
```

Explanation: This command provides detailed information about the `restaurant-data` volume, including its mount point on the host system. This is useful for understanding where the volume's data is physically stored.

Step 22: Run Container with Volume

```
docker run -d \  
--name restaurant-container1 \  
-p 80:80 \  
-v restaurant-data:/restaurant-data \  
restaurant:v1
```

Explanation: This command runs a new Docker container with the following options:

- `-d` : Runs the container in detached mode (in the background).
- `--name restaurant-container1` : Assigns the name `restaurant-container1` to the container.
- `-p 80:80` : Maps port 80 of the host to port 80 of the container, making the Nginx web server accessible from the host.
- `-v restaurant-data:/restaurant-data` : Mounts the `restaurant-data` Docker volume to the `/restaurant-data` directory inside the container. This is crucial for data persistence.
- `restaurant:v1` : Specifies the image to use for creating the container.

Step 23: Verify Running Container

```
docker ps
```

Explanation: This command lists all currently running Docker containers, confirming that `restaurant-container1` is active.

Step 24: Check Public IP

```
curl ifconfig.me
```

Explanation: This command retrieves the public IP address of the machine. This IP address will be used to access the Nginx web server running inside the Docker container from a web browser.

Browser Verification:

Open a web browser and navigate to `http://YOUR_PUBLIC_IP`. The restaurant webpage should be displayed, confirming that the Nginx server is running and accessible.

Interacting with the Container and Volume

Step 25: Login Inside Container

```
docker exec -it restaurant-container1 bash
```

Explanation: This command allows you to execute a command inside a running container.

- `docker exec` : Executes a command in a running container.
- `-it` : Allocates a pseudo-TTY and keeps stdin open, allowing for interactive shell access.
- `restaurant-container1` : The name of the target container.
- `bash` : The command to execute inside the container, which opens a Bash shell.

Step 26: Go to Volume Directory

```
cd /restaurant-data
```

Explanation: Once inside the container, this command changes the current directory to `/restaurant-data`, which is the mount point for the Docker volume.

Step 27: Create Folder

```
mkdir backup
```

Explanation: This command creates a new directory named `backup` inside the `/restaurant-data` volume. This folder will be used to store files that demonstrate volume persistence.

Step 28: Enter Folder

```
cd backup
```

Explanation: This command changes the current directory to the newly created `backup` folder.

Step 29: Create Files

```
echo "Hello Docker Volume" > file1.txt  
echo "Persistent Data" > file2.txt  
echo "Docker Volume Demo" > file3.txt
```

Explanation: These commands create three text files (`file1.txt`, `file2.txt`, `file3.txt`) within the `backup` directory and write specific content into each. These files will be used to verify data persistence.

Step 30: Verify Files

```
ls -l
```

Explanation: This command lists the contents of the current directory (`backup`) in a long format, showing the newly created files and their permissions, size, and modification dates.

Output:

```
file1.txt  
file2.txt  
file3.txt
```

Step 31: Read Files

```
cat file1.txt  
cat file2.txt  
cat file3.txt
```

Explanation: These commands display the content of each created file, confirming that the data was written correctly.

Output:

```
Hello Docker Volume  
Persistent Data  
Docker Volume Demo
```

Step 32: Exit Container

```
exit
```

Explanation: This command exits the interactive Bash shell session within the container, returning to the host's shell.

Container Lifecycle and Data Persistence Verification

Step 33: Stop Container

```
docker stop restaurant-container1
```

Explanation: This command gracefully stops the running `restaurant-container1` container.

Step 34: Remove Container

```
docker rm restaurant-container1
```

Explanation: This command removes the stopped `restaurant-container1` container. At this point, if the data were not in a volume, it would be lost.

Step 35: Create New Container Using Same Volume

```
docker run -d \  
--name restaurant-container2 \  
-p 80:80 \  
-v restaurant-data:/restaurant-data \  
restaurant:v1
```

Explanation: This command creates and runs a new container named `restaurant-container2` from the same `restaurant:v1` image, and importantly, it mounts the *same* `restaurant-data` volume to `/restaurant-data`. This demonstrates that the volume persists independently of the container.

Step 36: Login into New Container

```
docker exec -it restaurant-container2 bash
```

Explanation: This command provides interactive shell access to the newly created `restaurant-container2` container.

Step 37: Go to Backup Folder

```
cd /restaurant-data/backup
```

Explanation: This command navigates to the `backup` directory within the mounted volume inside the new container.

Step 38: Verify Files

```
ls -l
```

Explanation: This command lists the contents of the `backup` directory in the new container. The presence of `file1.txt`, `file2.txt`, and `file3.txt` here confirms that the data persisted even after the original container was stopped and removed.

Expected Output:

```
total 12

file1.txt
file2.txt
file3.txt
```

Step 39: Read Files

```
cat file1.txt  
cat file2.txt  
cat file3.txt
```

Explanation: These commands display the content of the files within the `backup` directory in the new container. The output should be the same as when the files were created in the first container, definitively proving Docker volume persistence.

Expected Output:

```
Hello Docker Volume  
Persistent Data  
Docker Volume Demo
```

Conclusion

This practical demonstrates the fundamental concept of Docker Volume Persistence. By using Docker volumes, data generated and used by containers can be stored independently of the container's lifecycle. This ensures that data remains intact even if containers are stopped, removed, or replaced, which is crucial for stateful applications and data management in a Dockerized environment.